ARQUITECTURA DE SOFTWARE

LABORATORIO NRO 1: Introducción a SpringBoot

Se realizará una aplicación que simule las transacciones de un banco que permita a los usuarios realizar movimientos entre cuentas, y crear nuevos usuarios, así como consultar el histórico de las transacciones.

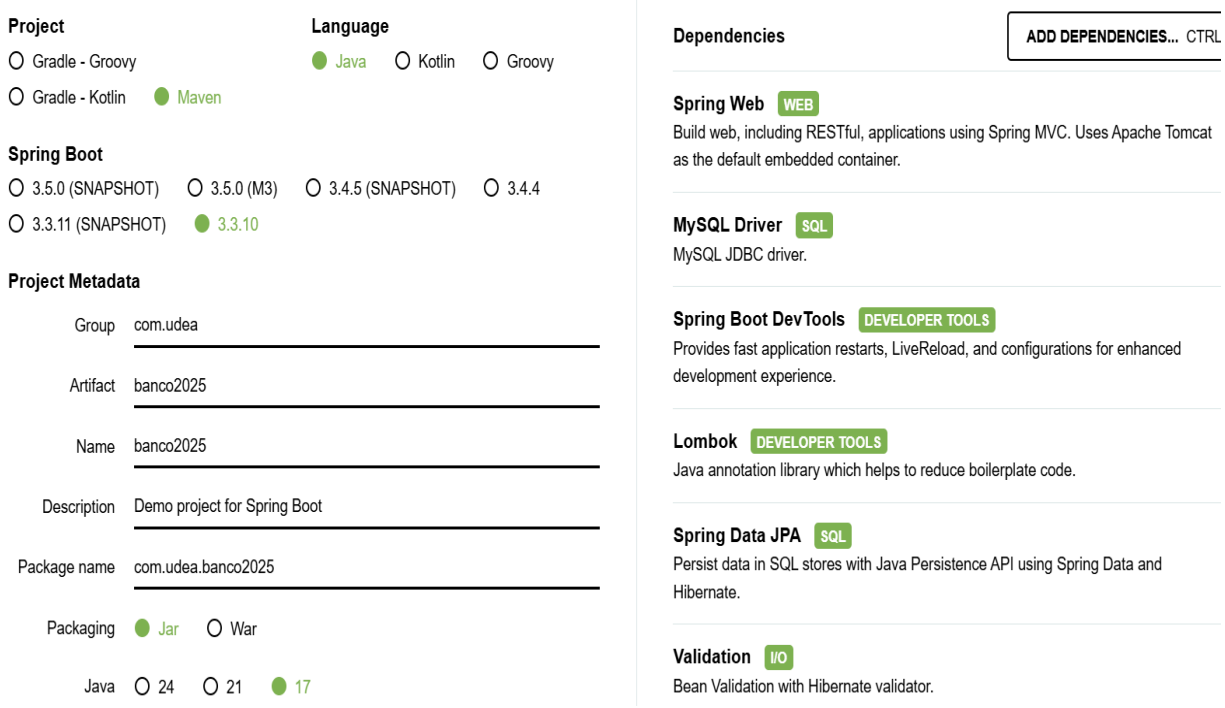
La aplicación consta de dos partes principales: el backend, que proporciona una API REST, y el frontend, que permite a los usuarios interactuar con la aplicación a través de una interfaz de usuario intuitiva y amigable.

Herramientas Empleadas

A. IDE IntelliJ y Visual Studio Code
B. Web Browser (Firefox, Chrome).
C. Spring Boot.
D. JDK 11 u 17
E. NodeJS
F. React JS
G. Axios.
H. Maven 3.5 o superior.

1. Creación del Proyecto Spring Boot

1. **Utilizar Spring Initializr:**
   - Creamos el proyecto en Spring Boot a través de Initializr accediendo al siguiente link https://start.spring.io/
   - Configura el proyecto con las siguientes opciones:
     - **Project:** Maven Project
     - **Language:** Java
     - **Spring Boot:** Selecciona la versión estable más reciente.
     - **Project Metadata:**
       - **Group:** com.ejemplo.banco
       - **Artifact:** banco2025
       - **Name:** banco2025
       - **Description:** Proyecto de ejemplo para una aplicación bancaria
       - **Package name:** com.ejemplo.banco
     - **Packaging:** Jar
     - **Java:** Selecciona la versión compatible con tu entorno (por ejemplo, 17).
2. **Añadir Dependencias:**
   - **Spring Web:** Para construir aplicaciones web, incluyendo RESTful.
   - **Spring Data JPA:** Para interactuar con bases de datos utilizando JPA.
   - **MySQL Driver:** Para conectar con la base de datos MySQL.
   - **Spring Boot DevTools:** Para facilitar el desarrollo con recarga automática.
   - **Lombok:** Para reducir el código repetitivo en las clases Java.



3. **Generar y Descargar el Proyecto:**
   - Haz clic en "Generate" para descargar el proyecto en formato ZIP.
   - Descomprime el archivo y abre el proyecto en tu IDE preferido (por ejemplo, IntelliJ IDEA o Eclipse).

2. Configuración de la Base de Datos MySQL

1. **Instalar MySQL:**
   - Descarga e instala MySQL desde https://dev.mysql.com/downloads/ .
   - Durante la instalación, configura un usuario y una contraseña que se utilizarán más adelante.
2. **Crear la Base de Datos:**
   - Accede al cliente de MySQL utilizando el usuario y contraseña configurados:

mysql -u tu_usuario -p

   - Crea una nueva base de datos para la aplicación:

CREATE DATABASE banco2025;

   - Opcionalmente, crea un usuario específico para la aplicación y otórgale los permisos necesarios:

CREATE USER 'banco_user'@'localhost' IDENTIFIED BY 'tu_contraseña';
GRANT ALL PRIVILEGES ON banco2025.* TO 'banco_user'@'localhost';
FLUSH PRIVILEGES;

3. **Archivo POM.XML**

El archivo principal de configuración de MAVEN del proyecto POM.xml tendrá la siguiente estructura:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
      <modelVersion>4.0.0</modelVersion>
      <parent>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-parent</artifactId>
            <version>3.3.10</version>
            <relativePath/> <!-- lookup parent from repository -->
```

```
</parent>
<groupId>com.udea</groupId>
<artifactId>banco2025</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>banco2025</name>
<description>Demo project for Spring Boot</description>
<url/>
<licenses>
      <license/>
</licenses>
<developers>
      <developer/>
</developers>
<scm>
      <connection/>
      <developerConnection/>
      <tag/>
      <url/>
</scm>
<properties>
      <java.version>17</java.version>
  <mapstruct.version>1.5.5.Final</mapstruct.version>
</properties>
<dependencies>
      <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
      </dependency>
      <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
      </dependency>


  <!-- Spring Boot Validation -->
  <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

      <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
      </dependency>
      <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
      </dependency>
      <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>1.18.30</version>  <!-- Usa la versión más reciente -->
            <scope>provided</scope>
      </dependency>
```

```
        <dependency>
              <groupId>com.fasterxml.jackson.core</groupId>
              <artifactId>jackson-databind</artifactId>
              <version>2.15.0</version>
        </dependency>

    <!-- MapStruct (DTO Mapping) -->
    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct</artifactId>
        <version>${mapstruct.version}</version>
    </dependency>
    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct-processor</artifactId>
        <version>${mapstruct.version}</version>
        <scope>provided</scope>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok-mapstruct-binding</artifactId>
        <version>0.2.0</version>
    </dependency>

        <dependency>
              <groupId>org.springframework.boot</groupId>
              <artifactId>spring-boot-starter-test</artifactId>
              <scope>test</scope>
        </dependency>
  </dependencies>

  <build>
        <plugins>
              <plugin>
                    <groupId>org.apache.maven.plugins</groupId>
                    <artifactId>maven-compiler-plugin</artifactId>
                    <configuration>
                          <annotationProcessorPaths>
                                <path>
                                      <groupId>org.projectlombok</groupId>
                                      <artifactId>lombok</artifactId>
                                </path>

                     <path>
                         <groupId>org.mapstruct</groupId>
                         <artifactId>mapstruct-processor</artifactId>
                         <version>${mapstruct.version}</version>
                     </path>
                  </annotationProcessorPaths>
                    </configuration>
              </plugin>
              <plugin>
                    <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-maven-plugin</artifactId>
                    <configuration>
```

```
                                <excludes>
                                      <exclude>
                                            <groupId>org.projectlombok</groupId>
                                            <artifactId>lombok</artifactId>
                                      </exclude>
                                </excludes>
                          </configuration>
                    </plugin>
              </plugins>
        </build>

</project>
```

4. Configuración de application.properties

En el archivo src/main/resources/application.properties, configura los parámetros de conexión a la base de datos MySQL:

```
spring.application.name=banco2025
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/banco?useSSL=false&allowPublicKey
Retrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.springframework=INFO
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
spring.transaction.default-timeout=30
spring.transaction.rollback-on-commit-failure=true
spring.mvc.contentnegotiation.favor-path-extension=false
spring.mvc.contentnegotiation.favor-parameter=false
spring.mvc.contentnegotiation.media-types.json=application/json
```

Configuraciones generales de la aplicación

spring.application.name=banco2025

- Define el nombre de la aplicación como banco2025. Este valor es útil para la identificación de la aplicación en logs y monitoreo.

Configuraciones del servidor

server.port=8080

- Especifica el puerto en el que se ejecutará la aplicación. En este caso, es el 8080.

Configuraciones de la base de datos

spring.datasource.url=jdbc:mysql://localhost:3306/banco?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC

- Define la URL de conexión a la base de datos MySQL.
    - localhost:3306: Dirección del servidor MySQL en el puerto 3306.
    - banco: Nombre de la base de datos.
    - useSSL=false: Indica que no se usará SSL en la conexión.
    - allowPublicKeyRetrieval=true: Permite la recuperación de claves públicas para autenticación cuando se usa caching_sha2_password.
    - serverTimezone=UTC: Configura la zona horaria del servidor.

spring.datasource.username=root
spring.datasource.password=root

- Define el usuario y la contraseña de acceso a la base de datos.

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

- Especifica el driver JDBC que se usará para conectar con MySQL.

Configuraciones de JPA e Hibernate

spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect

- Define el dialecto de Hibernate para MySQL 8, optimizando la generación de consultas SQL.

spring.jpa.hibernate.ddl-auto=update

- Especifica cómo Hibernate debe manejar el esquema de la base de datos:
    - update: Modifica la estructura de la base de datos sin borrar datos existentes.
    - Otros valores posibles: none, create, create-drop, validate.

spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

- show-sql=true: Hace que Spring Boot muestre en consola las consultas SQL ejecutadas.
- format_sql=true: Formatea las consultas SQL para una mejor legibilidad en los logs.

Configuraciones de logging

logging.level.org.springframework=INFO
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE

- Define los niveles de log para diferentes paquetes:
    - org.springframework=INFO: Muestra solo información relevante de Spring Boot.
    - org.hibernate.SQL=DEBUG: Muestra las consultas SQL generadas por Hibernate.
    - org.hibernate.type.descriptor.sql.BasicBinder=TRACE: Muestra los valores de los parámetros en las consultas SQL.

Configuraciones de transacciones

spring.transaction.default-timeout=30
spring.transaction.rollback-on-commit-failure=true

- default-timeout=30: Establece el tiempo máximo (en segundos) para que una transacción se complete antes de ser abortada.
- rollback-on-commit-failure=true: Si ocurre un error al confirmar una transacción, se revierte automáticamente.

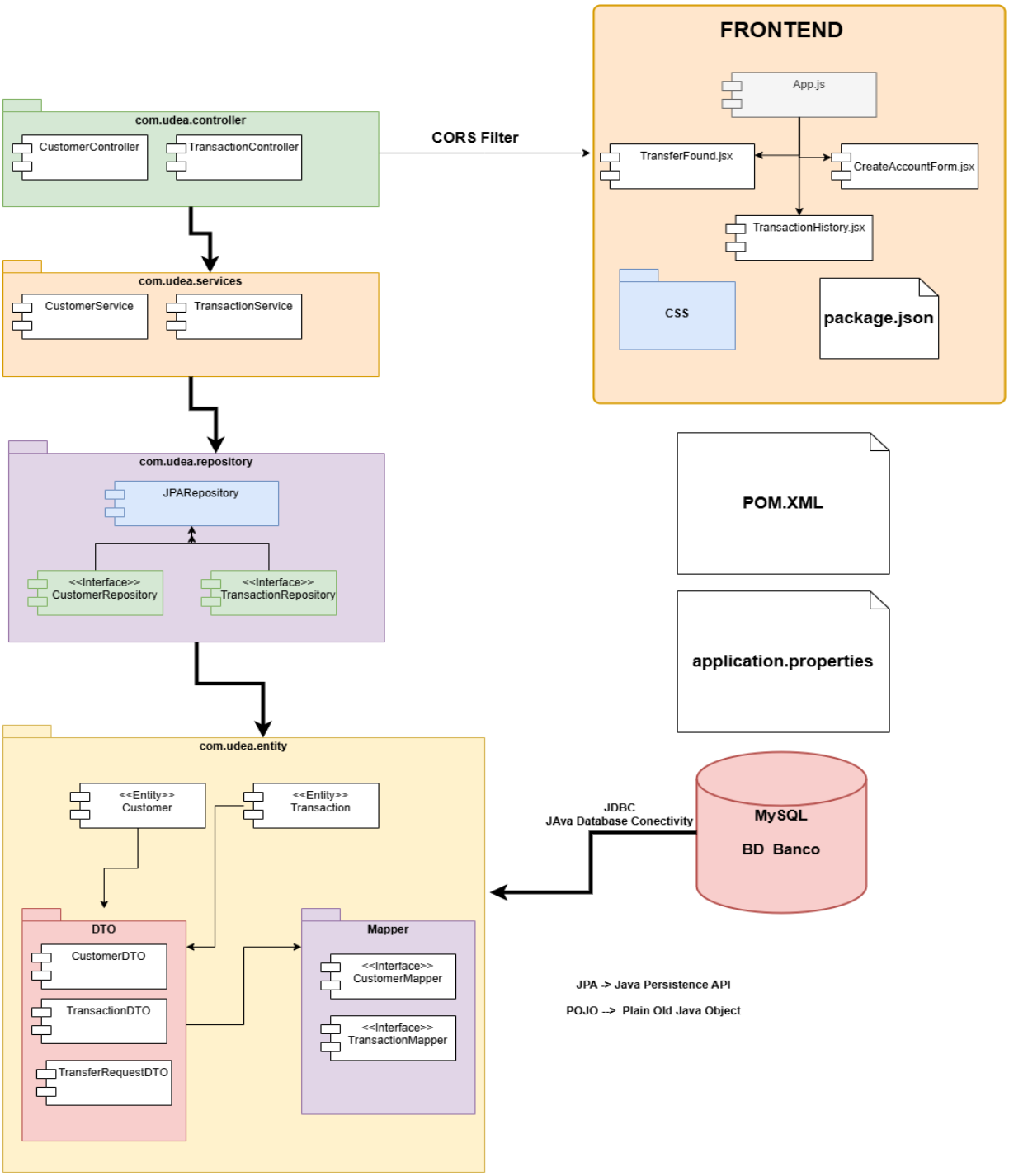
Configuraciones de Spring MVC

spring.mvc.contentnegotiation.favor-path-extension=false
spring.mvc.contentnegotiation.favor-parameter=false
spring.mvc.contentnegotiation.media-types.json=application/json

- favor-path-extension=false: Evita el uso de extensiones en las rutas (.json, .xml) para determinar el tipo de respuesta.
- favor-parameter=false: No permite definir el formato de respuesta mediante parámetros en la URL (?format=json).
- media-types.json=application/json: Define que el formato de salida por defecto será JSON.

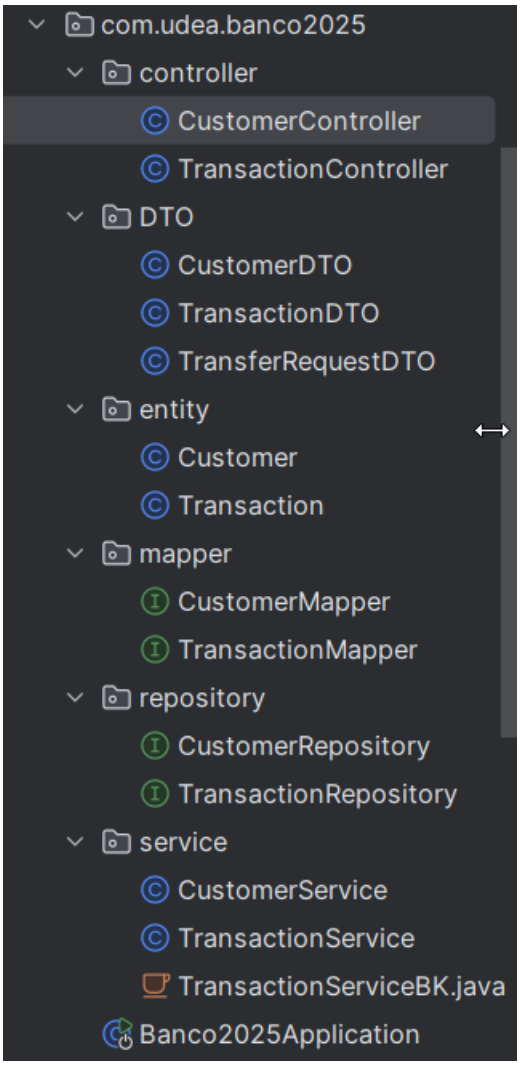
4. Implementación de la Arquitectura por Capas

Organiza el proyecto siguiendo una arquitectura por capas para mejorar la mantenibilidad y escalabilidad:



- **Modelo (Model):** Contiene las clases que representan las entidades, los DTOs y los Mappers de la aplicación.
- **Repositorio (Repository):** Interfaces que gestionan la comunicación y los métodos transaccionales con la base de datos.
- **Servicio (Service):** Contiene la lógica de negocio de la aplicación.
- **Controlador (Controller):** Maneja las solicitudes HTTP y las respuestas.

La estructura del proyecto quedará así:



4.1. Capa de Modelo
Crea la clase Transaction que representa una transacción bancaria:

Transaction.java

```
package com.ejemplo.banco.model;
import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;
@Data
@Entity
@Table(name = "transactions")
public class Transaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "sender_account_number", nullable = false)
    private String senderAccountNumber;
```

```
    @Column(name = "receiver_account_number", nullable = false)
    private String receiverAccountNumber;

    @Column(nullable = false)
    private Double amount;

    @Column(nullable = false)
    private LocalDateTime timestamp;
}
```

Explicación:

- @Data: Anotación de Lombok que genera automáticamente los métodos getter, setter, toString, equals y hashCode.
- @Entity: Indica que esta clase es una entidad JPA que se mapeará a una tabla en la base de datos.
- @Table(name = "transactions"): Especifica el nombre de la tabla en la base de datos.
- @Id: Especifica el campo que será la clave primaria.
- @GeneratedValue(strategy = GenerationType.IDENTITY): Define la estrategia de generación automática de valores para la clave primaria.
- @Column: Especifica detalles de las columnas en la base de datos, como el nombre y si son obligatorias.

Customer.java

```
package com.udea.bancoudea.entity;

import com.fasterxml.jackson.annotation.JsonCreator;
import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.persistence.*;

@Entity
@Table(name = "customers")
public class Customer {
    // POJO

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(unique = true, nullable = false)
    private String accountNumber;
    @Column(nullable = false, length = 50)
    private String firstName;
    @Column(nullable = false, length = 50)
    private String lastName;
    @Column(nullable = false)
    private Double balance;

    public Customer() {
    }
    @JsonCreator
    public Customer(@JsonProperty("id") Long id, @JsonProperty("accountNumber")
String accountNumber, @JsonProperty("firstName") String firstName,
@JsonProperty("lastName")  String lastName, @JsonProperty("balance")  Double
balance) {
        this.id = id;
        this.accountNumber = accountNumber;
```

```
        this.firstName = firstName;
        this.lastName = lastName;
        this.balance = balance;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public Double getBalance() {
        return balance;
    }

    public void setBalance(Double balance) {
        this.balance = balance;
    }
}
```

4.2. Capa de Repositorio

Crea la interfaz TransactionRepository que extiende JpaRepository para interactuar con la base de datos:

`TransactionRepository.java`

```
package com.ejemplo.banco.repository;

import com.ejemplo.banco.model.Transaction;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface TransactionRepository extends JpaRepository<Transaction, Long> {
    List<Transaction>        findBySenderAccountNumberOrReceiverAccountNumber(String
senderAccountNumber, String receiverAccountNumber);
}
```

Explicación:

- @Repository: Indica que esta interfaz es un componente de acceso a datos.
- JpaRepository: Proporciona métodos CRUD y de paginación para la entidad Transaction.

CustomerRepository.java

```
package com.udea.bancoudea.repository;

import com.udea.bancoudea.entity.Customer;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    Optional<Customer> findByAccountNumber(String accountNumber);
}
```

4.3. Capa de Servicio

La capa de servicio contiene la lógica de negocio de la aplicación. A continuación, se muestra la implementación de la clase TransactionService:

TransactionService.java

```
package com.udea.bancoudea.service;

import com.udea.bancoudea.DTO.TransactionDTO;
import com.udea.bancoudea.entity.Customer;
import com.udea.bancoudea.entity.Transaction;
import com.udea.bancoudea.repository.CustomerRepository;
import com.udea.bancoudea.repository.TransactionRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class TransactionService {

    @Autowired
    private TransactionRepository transactionRepository;

    @Autowired
    private CustomerRepository customerRepository;
```

```
    public TransactionDTO transferMoney(TransactionDTO transactionDTO) {
        //validar que los numeros de cuenta no sean nulos
        if(transactionDTO.getSenderAccountNumber()==null ||
transactionDTO.getReceiverAccountNumber()==null){
            throw new IllegalArgumentException("Sender Account Number or Receiver
Account Number cannot be null");
        }

        //Buscar los clientes por numero de cuenta
        Customer sender =
customerRepository.findByAccountNumber(transactionDTO.getSenderAccountNumber())
                .orElseThrow(()-> new IllegalArgumentException("Sender Account
Number not found"));

        Customer receiver =
customerRepository.findByAccountNumber(transactionDTO.getReceiverAccountNumber())
                .orElseThrow(()-> new IllegalArgumentException("Receiver Account
Number not found"));

        //Validar que el remitente tenga saldo suficiente
        if(sender.getBalance() < transactionDTO.getAmount()){
            throw new IllegalArgumentException("Sender Balance not enough");
        }

        //realiza la transferencia
        sender.setBalance(sender.getBalance() - transactionDTO.getAmount());
        receiver.setBalance(receiver.getBalance() + transactionDTO.getAmount());

        //Guardar los cambios en las cuentas
        customerRepository.save(sender);
        customerRepository.save(receiver);

        //Crear y guardar la transaccion
        Transaction transaction = new Transaction();
        transaction.setSenderAccountNumber(sender.getAccountNumber());
        transaction.setReceiverAccountNumber(receiver.getAccountNumber());
        transaction.setAmount(transactionDTO.getAmount());
        transaction= transactionRepository.save(transaction);

        //Devolver la transaccion creada como un DTO
        TransactionDTO  savedTransaction = new TransactionDTO();
        savedTransaction.setId(transaction.getId());

savedTransaction.setSenderAccountNumber(transaction.getSenderAccountNumber());

savedTransaction.setReceiverAccountNumber(transaction.getReceiverAccountNumber());
        savedTransaction.setAmount(transaction.getAmount());
        return savedTransaction;

    }

    public List<TransactionDTO> getTransactionsForAccount(String accountNumber) {
        List<Transaction> transactions =
transactionRepository.findBySenderAccountNumberOrReceiverAccountNumber(accountNumbe
r,accountNumber);
        return transactions.stream().map(transaction -> {
            TransactionDTO dto = new TransactionDTO();
```

```
            dto.setId(transaction.getId());
            dto.setSenderAccountNumber(transaction.getSenderAccountNumber());
            dto.setReceiverAccountNumber(transaction.getReceiverAccountNumber());
            dto.setAmount(transaction.getAmount());
            return dto;
        }).collect(Collectors.toList());
    }

}
```

Explicación:

- @Service: Marca la clase como un componente de servicio en Spring.
- TransactionService: Proporciona métodos para obtener todas las transacciones, buscar una transacción por ID y hacer transferencias.

CustomerService.java

```
package com.udea.bancoudea.service;

import com.udea.bancoudea.DTO.CustomerDTO;
import com.udea.bancoudea.entity.Customer;
import com.udea.bancoudea.mapper.CustomerMapper;
import com.udea.bancoudea.repository.CustomerRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class CustomerService {

    private final CustomerRepository customerRepository;
    private final CustomerMapper customerMapper;

    @Autowired
    public CustomerService(CustomerRepository customerRepository, CustomerMapper
customerMapper) {
        this.customerRepository = customerRepository;
        this.customerMapper = customerMapper;
    }

    public List<CustomerDTO> getAllCustomer(){
        return customerRepository.findAll().stream()
                .map(customerMapper::toDTO).toList();
    }

    public CustomerDTO getCustomerById(Long id){
        return customerRepository.findById(id).map(customerMapper::toDTO)
                .orElseThrow(()->new RuntimeException("Cliente no encontrado"));
    }

    public CustomerDTO createCustomer(CustomerDTO customerDTO){
        Customer customer = customerMapper.toEntity(customerDTO);
        return customerMapper.toDTO(customerRepository.save(customer));
    }
```

```
}
```

4.4. Capa de Controlador

La capa de controlador maneja las solicitudes HTTP y define los endpoints de la aplicación. A continuación, se muestra la implementación del TransactionController:

CustomerController.java

```
package com.udea.bancoudea.controller;

import com.udea.bancoudea.DTO.CustomerDTO;
import com.udea.bancoudea.service.CustomerService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/customers")
public class CustomerController {
    private final CustomerService customerFacade;

    public CustomerController(CustomerService customerFacade) {
        this.customerFacade = customerFacade;
    }

    //Obtener todos los cliente
    @GetMapping
    public ResponseEntity<List<CustomerDTO>> getAllCustomers() {
        return ResponseEntity.ok(customerFacade.getAllCustomer());
    }

    //Obtener un cliente por un ID
    @GetMapping("/{id}")
    public ResponseEntity<CustomerDTO> getCustomerById(@PathVariable Long id) {
        return ResponseEntity.ok(customerFacade.getCustomerById(id));
    }

//Crear un nuevo cliente
    @PostMapping
    public ResponseEntity<CustomerDTO> createCustomer(@RequestBody CustomerDTO
customerDTO) {
        if(customerDTO.getBalance() == null) {
            throw new IllegalArgumentException("Balance cannot be null");
        }
        return ResponseEntity.ok(customerFacade.createCustomer(customerDTO));
    }
}
```

Explicación:

- @RestController: Indica que esta clase es un controlador REST.
- @RequestMapping("/api/customers"): Define la ruta base para todas las solicitudes relacionadas con los clientes del banco.
- @GetMapping, @PostMapping. Especifican los métodos HTTP para cada operación CRUD.
- @PathVariable: Extrae valores de la URL.

- @RequestBody: Indica que el cuerpo de la solicitud se convertirá en un objeto Java.

4. Uso de DTOs y Mappers en Spring Boot

En el desarrollo de aplicaciones, especialmente en arquitecturas multicapa, es fundamental controlar cómo se transfieren los datos entre las diferentes capas de la aplicación. Exponer directamente las entidades de la base de datos puede llevar a problemas de seguridad, acoplamiento y rendimiento. Para abordar estos desafíos, se emplean los DTOs (Data Transfer Objects), que son objetos simples diseñados para transportar datos entre procesos o capas sin contener lógica de negocio.

4.1. ¿Qué es un DTO?

Un DTO es una clase que encapsula datos y se utiliza para transferir información entre las capas de una aplicación. Su principal objetivo es reducir la cantidad de llamadas entre procesos y controlar qué datos se exponen al exterior, evitando revelar detalles internos de las entidades del dominio. Por ejemplo, al exponer una API REST, es recomendable devolver un DTO que contenga únicamente los campos necesarios, en lugar de la entidad completa con todos sus atributos.

A continuación, se da el código de los DTOs

CustomerDTO.java

```
package com.udea.bancoudea.DTO;

public class CustomerDTO {
    private Long id;
    private String firstName;
    private String lastName;
    private String accountNumber;
    private Double balance;

    public CustomerDTO() {
    }

    public CustomerDTO(Long id, String firstName, String lastName, String
accountNumber, Double balance) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
```

```
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    public Double getBalance() {
        return balance;
    }

    public void setBalance(Double balance) {
        this.balance = balance;
    }
}
```

TransactionDTO.java

```
import com.fasterxml.jackson.annotation.JsonInclude;

import java.time.LocalDateTime;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class TransactionDTO {
    private Long id;
    private String senderAccountNumber;
    private String receiverAccountNumber;
    private Double amount;
    private LocalDateTime timestamp;

    public TransactionDTO() {
    }

    public    TransactionDTO(Long    id,    String    senderAccountNumber,    String
receiverAccountNumber, Double amount, LocalDateTime timestamp) {
        this.id = id;
        this.senderAccountNumber = senderAccountNumber;
        this.receiverAccountNumber = receiverAccountNumber;
        this.amount = amount;
        this.timestamp = timestamp;
    }

    public Long getId() {
        return id;
```

```
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getSenderAccountNumber() {
        return senderAccountNumber;
    }

    public void setSenderAccountNumber(String senderAccountNumber) {
        this.senderAccountNumber = senderAccountNumber;
    }

    public String getReceiverAccountNumber() {
        return receiverAccountNumber;
    }

    public void setReceiverAccountNumber(String receiverAccountNumber) {
        this.receiverAccountNumber = receiverAccountNumber;
    }

    public Double getAmount() {
        return amount;
    }

    public void setAmount(Double amount) {
        this.amount = amount;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }
}
```

TransferRequestDTO.java

```
package com.udea.bancoudea.DTO;

public class TransferRequestDTO {
    private String senderAccountNumber;
    private String receiverAccountNumber;
    private Double amount;

    public TransferRequestDTO() {
    }

    public String getSenderAccountNumber() {
        return senderAccountNumber;
    }

    public void setSenderAccountNumber(String senderAccountNumber) {
        this.senderAccountNumber = senderAccountNumber;
```

```
    }

    public String getReceiverAccountNumber() {
        return receiverAccountNumber;
    }

    public void setReceiverAccountNumber(String receiverAccountNumber) {
        this.receiverAccountNumber = receiverAccountNumber;
    }

    public Double getAmount() {
        return amount;
    }

    public void setAmount(Double amount) {
        this.amount = amount;
    }
}
```

7.3. ¿Qué es un Mapper?

Un **Mapper** es un componente que se encarga de convertir entre diferentes tipos de objetos, como de una entidad a un DTO y viceversa. Esta conversión puede realizarse de manera manual o utilizando bibliotecas especializadas que automatizan el proceso, reduciendo el código repetitivo y minimizando errores.

7.4. Implementación de Mappers en Spring Boot

Existen diversas formas de implementar mapeos en Spring Boot:

- **Mapeo Manual:** Consiste en escribir métodos que conviertan explícitamente una entidad en un DTO y viceversa. Aunque es sencillo, puede volverse tedioso y propenso a errores en aplicaciones grandes.
- **Uso de Bibliotecas como ModelMapper o MapStruct:** Estas herramientas automatizan el proceso de mapeo, generando el código necesario durante la compilación o en tiempo de ejecución.

A continuación, se presenta el código de los mappers

CustomerMapper.java

```
package com.udea.bancoudea.mapper;

import com.udea.bancoudea.DTO.CustomerDTO;
import com.udea.bancoudea.entity.Customer;
import org.mapstruct.Mapper;
import org.mapstruct.factory.Mappers;

@Mapper(componentModel = "spring")
public interface CustomerMapper {
    CustomerMapper INSTANCE = Mappers.getMapper(CustomerMapper.class);
    CustomerDTO toDTO(Customer customer);
    Customer toEntity(CustomerDTO customerDTO);
}
```

Definición de la Interfaz

@Mapper(componentModel = "spring")
public interface CustomerMapper {

- @Mapper(componentModel = "spring"):
    - Indica que esta interfaz es un **mapeador** de MapStruct.

    - componentModel = "spring" permite que **Spring Boot** administre el mapper como un **bean**, por lo que puede ser inyectado con @Autowired.

Creación de una Instancia del Mapper
CustomerMapper INSTANCE = Mappers.getMapper(CustomerMapper.class);

- Mappers.getMapper(CustomerMapper.class):
    - Genera una **instancia automática** de CustomerMapper en tiempo de compilación.
    - Si componentModel = "spring", esta línea no es necesaria, porque Spring manejará la inyección.

Métodos de Conversión
CustomerDTO toDTO(Customer customer);
Customer toEntity(CustomerDTO dto);

- **toDTO(Customer customer)**: Convierte un objeto Customer en un CustomerDTO.
- **toEntity(CustomerDTO dto)**: Convierte un CustomerDTO en un Customer.

TransactionMapper.java

```
package com.udea.bancoudea.mapper;

import com.udea.bancoudea.DTO.TransactionDTO;
import com.udea.bancoudea.entity.Transaction;
import org.mapstruct.Mapper;
import org.mapstruct.factory.Mappers;

@Mapper
public interface TransactionMapper {
TransactionMapper INSTANCE= Mappers.getMapper(TransactionMapper.class);
TransactionDTO toDTO(Transaction transaction);
}
```

5. Ejecución de la Aplicación
Para ejecutar la aplicación, utiliza tu IDE para correr la clase principal que contiene el método main. Alternativamente, desde la línea de comandos, navega al directorio del proyecto y ejecuta:

./mvnw spring-boot:run
La aplicación estará disponible en http://localhost:8080.

6. Pruebas con Postman
Postman es una herramienta que permite probar APIs REST de manera sencilla. A continuación, se detallan las pruebas para cada endpoint de la aplicación.

6.1. Obtener todas las transacciones

- **Método:** GET
- **URL:** http://localhost:8080/api/transactions
- **Descripción:** Recupera todas las transacciones registradas.
- **Respuesta esperada:** Lista de transacciones en formato JSON.

6.2. Obtener una transacción por ID

- **Método:** GET
- **URL:** http://localhost:8080/api/transactions/{id}
- **Descripción:** Recupera una transacción específica por su ID.
- **Parámetros:**
    - id: ID de la transacción a recuperar.
- **Respuesta esperada:** Detalles de la transacción en formato JSON. Si no se encuentra, devuelve un estado 404.

6.3. Crear una nueva transacción

- **Método:** POST
- **URL:** http://localhost:8080/api/transactions
- **Descripción:** Crea una nueva transacción.
- **Cuerpo de la solicitud (JSON):**

```
{
 "senderAccountNumber": "123456789",
 "receiverAccountNumber": "987654321",
 "amount": 100.00
}
```

- **Respuesta esperada:** La transacción creada con su ID y marca de tiempo asignados.

6.4. Actualizar una transacción existente

- **Método:** PUT
- **URL:** http://localhost:8080/transactions/{id}
- **Descripción:** Actualiza los detalles de una transacción existente.
- **Parámetros:**
  - id: ID de la transacción a actualizar.
- **Cuerpo de la solicitud (JSON):**

```
{
 "senderAccountNumber": "123456789",
 "receiverAccountNumber": "987654321",
 "amount": 150.00
}
```

- **Respuesta esperada:** La transacción actualizada. Si no se encuentra, devuelve un estado 404.

6.5. Eliminar una transacción

- **Método:** DELETE
- **URL:** http://localhost:8080/transactions/{id}
- **Descripción:** Elimina una transacción por su ID.
- **Parámetros:**
  - id: ID de la transacción que se desea eliminar.
- **Respuesta esperada:** Código de estado HTTP 204 (No Content) indicando que la eliminación fue exitosa.

7. Consideraciones Finales

- **Manejo de Errores:** Es recomendable implementar un manejo adecuado de excepciones en los controladores para proporcionar respuestas más informativas en caso de errores, como transacciones no encontradas o datos inválidos.
- **Validación de Datos:** Utiliza anotaciones de validación en tus entidades y métodos para asegurar que los datos recibidos cumplen con los criterios esperados antes de procesarlos o almacenarlos en la base de datos.
- **Seguridad:** Considera implementar medidas de seguridad, como autenticación y autorización, para proteger tus endpoints y asegurar que solo usuarios autorizados puedan acceder o modificar los datos.
- **Documentación:** Herramientas como Swagger pueden ser útiles para documentar tus APIs REST, facilitando su comprensión y uso por parte de otros desarrolladores o equipos.

Link de GitHub del proyecto: https://github.com/diegobotia/lab12026p

Ejercicio de entrega obligatoria

Fecha de entrega 14 de Marzo de 2026

Desarrollar un frontend básico que permita consumir las rutas definidas desde los controladores. Mínimo debe tener 3 vistas (Consultar clientes, Realizar transferencia de dinero entre cuentas y crear una tabla con el histórico de las transacciones por cada cliente.

Nota: Puede agregar opcionalmente la funcionalidad para actualizar o borrar la información de los clientes.